

TaskTinyVLMs: Embodied Task Planning via Parameter-Optimized Vision-Language Model Fine-Tuning

Xiwei Zhao^{1,2,*}

¹ByteDance Seed, ²Aalto University

*Work done at ByteDance Seed

Abstract

This study addresses the critical challenge of deploying large vision-language models (VLMs) for real-time embodied task planning. While VLMs demonstrate strong multimodal reasoning capabilities, they consistently fail to achieve high-accuracy task planning or real-time inference in embodied scenarios requiring dynamic environment interaction. To bridge this gap, we propose an efficient fine-tuning framework for lightweight VLMs (e.g., Qwen2-VL-7B) on the EmbodiedBench benchmark. We introduce an automated data collection framework where GPT-4o with CoT-optimized prompts generates complexity-tiered tasks and related evaluation functions to gather environment-interaction trajectories; and a parameter optimization analysis through systematic experimentation that determines optimal training configurations, enabling lightweight VLMs to match commercial model performance. Extensive experiments demonstrate that our fine-tuned 7B model achieves near-parity with GPT-4o while drastically reducing inference overhead.

Date: June 21, 2025

Correspondence: Author at zhaoxiwei@bytedance.com

1 Introduction

In the current era of booming artificial intelligence technology, Embodied AI, as a key area for enabling agents to make autonomous decisions and take action in the real physical world, is receiving increasing attention. Vision-Language Models (VLMs), with their powerful multimodal understanding and generation capabilities, serve as a bridge connecting visual perception and language instructions, showing immense potential for application in embodied intelligence tasks. The emergence of the EmbodiedBench[1] dataset has established a configuration system for multimodal task scenarios and a quantitative evaluation metric system for task completion. It also proposes a task planning instruction generation framework for VLMs, providing an important foundation and evaluation benchmark for research in embodied intelligence.

However, as the application scenarios for embodied intelligence continue to expand and become more complex, existing technologies face new challenges in dataset generation and model deployment. In terms of dataset generation, the traditional manual method of constructing task instruction pipelines is inefficient and struggles to meet the demands of large-scale, diverse task scenarios. There is an urgent need for automated dataset generation technology to improve the efficiency and validity of task instruction generation, adapting to the requirements of complex multimodal environmental parameters and strict task objective constraints.

In terms of model application, when used for real-time decision-making in robots, VLMs often require substantial computational resources and time for inference, making it difficult to meet the demands for real-time performance and lightweight deployment. Therefore, it is necessary to design more efficient fine-tuning strategies and inference pipelines tailored to the actual application scenarios of robots, achieving hierarchical decision modeling from environmental perception to task planning.

This study aims to address the shortcomings of existing technologies in embodied intelligence tasks by conducting extended research based on the achievements of the EmbodiedBench dataset, with the goal of promoting the practical application of VLMs in the field of embodied intelligence. Specifically, it includes the following two research objectives:

First, to construct an automated generation framework for task instruction pipelines to achieve the generation of reasonable task instructions under the constraints of multimodal environmental parameters and task objectives. By researching automated dataset generation technology, this study will solve the inefficiency of traditional manual generation methods, improve the quality and diversity of the dataset, and provide VLMs with richer training data that better fits real-world scenarios, thereby enhancing the model’s ability to understand and plan for complex tasks.

Second, to meet the needs of real-time robot decision-making, this study will design a few-shot fine-tuning strategy based on prompt engineering and construct an end-to-end inference framework that includes real-time environmental feedback and visual input. This will achieve hierarchical decision modeling from environmental perception to task planning. By performing lightweight fine-tuning on the VLM, the model’s dependence on computational resources will be reduced and inference speed will be increased, allowing it to better adapt to applications in the dynamic, real-time environments of robots. This provides more efficient and reliable technical support for autonomous robot decision-making and action.

The main contributions of this research are as follows:

- **Proposed and implemented an automated pipeline for small model fine-tuning:** A complete automated pipeline was researched and constructed, covering the three key stages of data generation, data processing, and model fine-tuning, which significantly improved the efficiency and reproducibility of small model fine-tuning.
- **Systematically investigated the impact mechanism of fine-tuning parameters on model performance:** Through rigorous experimental design and comparative analysis, this study deeply revealed the key roles and patterns of specific fine-tuning parameters (such as learning rate, data ratio, loss function weights, etc.) in improving the performance of small models, providing an empirical basis for parameter optimization.
- **Achieved a significant breakthrough in the performance of a small-scale model:** The fine-tuned small model performance was successfully elevated to a level comparable to that of large commercial models (such as GPT-4o), verifying the technical feasibility and cost-effectiveness of using small models to achieve state-of-the-art performance in specific tasks or scenarios.

2 Approach

The EmbodiedBench[1] dataset establishes a comprehensive multimodal task scenario configuration system and a quantified task completion evaluation framework, along with a VLM-oriented task planning instruction generation methodology. In this research, two core contributions have been extended in the foundation of EmbodiedBench. An integrated dataset generation pipeline has been presented, in which tasks instructions and related evaluation functions are automatically generated. Besides, the finetuning and online inference pipeline has also been constructed. For the real-time decision-making requirements of robots, a few-shot fine-tuning strategy based on prompt engineering is designed, and an end-to-end reasoning framework incorporating real-time environmental feedback and visual input is constructed to achieve hierarchical decision-making modeling from environmental perception to task planning.

2.1 Data generation

EmbodiedBench offers datasets across four distinct scenarios with varying task complexities. These datasets encompass the scene configurations for the current episode, task objectives, and evaluation functions to gauge task completion. Additionally, EmbodiedBench integrates task planning modules powered by various Vision-Language Models (VLMs) into multiple simulators, and the associated paper establishes experimental baselines. For instance, in the base task scenario, the Qwen2-VL-72B model achieves a success rate of 60%. Building upon EmbodiedBench, our research constructs a data generation pipeline that includes updating task scenarios, refining task objectives, automatically generating task evaluation functions, and creating fine-tuning datasets.

2.1.1 Task generation

This study employs GPT-4o to generate task instructions with varying difficulty levels. Specifically, by semantically decomposing the task space of robotic embodied intelligence, task difficulty is categorized into multiple hierarchies: basic operations, ambiguous language instructions, complex language instructions, etc. As illustrated in Fig. 1 and Fig. 2, inputting task difficulty parameters into GPT-4o dynamically generates task instructions that align with different difficulty gradients.



Figure 1 Basic task example: find an orange and places it on the TV cabinet.



Figure 2 Ambiguous instruction: A piece of fruit is needed on the living room table.

2.1.2 Evaluation functions generation

This section also employs GPT-4o to generate evaluation functions corresponding to task objectives. As shown in the Fig. 3 below, this module takes task objectives as input, combines the physical properties and spatial relationships of the simulation scenario, and automatically generates evaluation functions. After each step is executed in the simulation, the system invokes the evaluation function to monitor the state of objects in the scenario. When the current state is detected to meet the preset objectives, a success command is triggered, the current scenario state and historical trajectory are recorded, and then the execution of the next task is initiated.

```
User instruction: find a banana and put it on the table 1
Goal updated: {'expr_type': 'AND', 'quantifier': 'EXISTS',
'inputs': [{'name': 'X', 'expr_type': 'banana'}],
'sub_exprs': ['on_top(X, receptacle_aabb_Tb11_Top1_frl_apartment_table_01)',
'not_holding()']}
```

Figure 3 User instruction and task goal pair example

2.1.3 Training dataset generation

This study optimizes the prompts provided in EmbodiedBench using Chain-of-Thought (CoT) reasoning. By incorporating reflection and reasoning steps, it outputs higher-quality action information. After CoT

reasoning optimization, the task success rate of GPT-4o has significantly improved in task scenarios with different difficulty levels as shown in Table 1.

Table 1 CoT optimization results comparison

Task levels	base	common sense	complex instruction
EmbodiedBench baseline	60%	20%	22%
After CoT optimization	86%	38%	42%

Based on the CoT reasoning-optimized natural language instruction prompts, closed-loop simulation experiments are conducted using the GPT-4o model. The system collects multi-dimensional state data (including action execution logs and environmental feedback). According to the collected action execution logs, training datasets of different patterns are generated:

- Pattern 1: Input language instructions and environmental feedback information, output reasoning information and action sequences.
- Pattern 2: Input language instructions, environmental feedback information, and multiple examples, output reasoning information and action sequences.
- Pattern 3: Input language instructions and environmental feedback information, output action sequences.
- Pattern 4: Input language instructions, environmental feedback information, and multiple examples, output action sequences.

2.2 Training

This study employs Supervised Fine-Tuning (SFT) technology, and the training framework adopts LlamaFactory [3].

2.2.1 Dataset statistics

In three tasks with different difficulty levels, approximately 300 pieces of data were collected. Training datasets in three formats were generated based on whether system prompts and examples were added to input items and whether reasoning generation was added to output items, as shown in the Table 2:

Table 2 Details of the generated training datasets.

ID	Camera Views	Model	Quantity	Input Includes Failed Steps	Input Includes System Prompt & Examples	Input Includes History Steps	Output Includes Reasoning
v0	2	gpt-4o	327	Yes	No	Yes	Yes
v1	2	gpt-4o	295	Yes	No	Yes	No
v2	2	gpt-4o	295	Yes	Yes	Yes	No

2.2.2 Training parameters

This study uses an L40 host and employs LORA training technology. Other parameters are shown in the Table. 3.

The purpose of this study is to enable VLMs to generate more precise and robust action plans for complex and dynamic embodied task scenarios. This training objective fundamentally differs from the generalized training paradigm of traditional large model backbones. Meanwhile, due to the small scale of task planning datasets, long-term iterative training is difficult to support. Therefore, this study adopts a phased and multi-round progressive training strategy. By setting different numbers of training rounds for comparative experiments, we systematically explore the model’s performance at each training stage, effectively avoiding

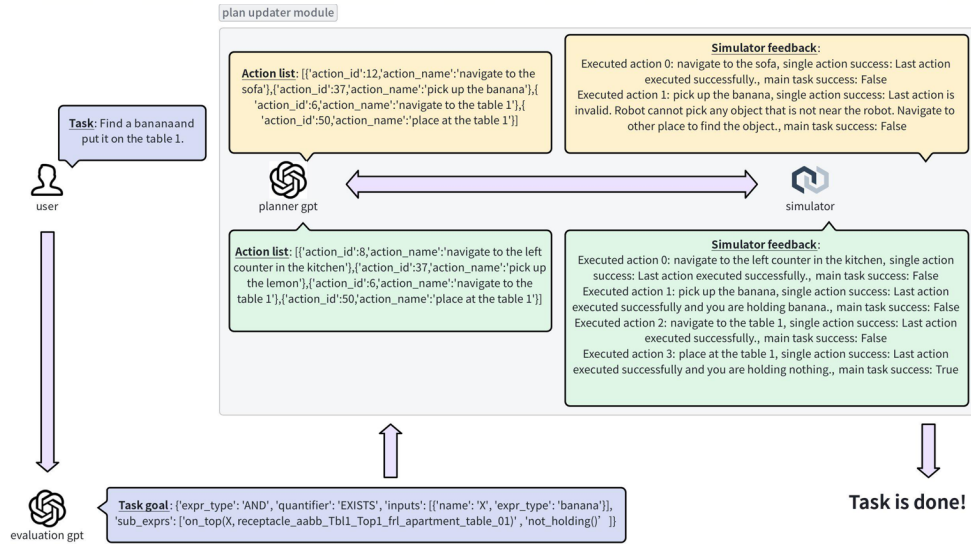
Table 3 Training Parameters

Parameter	Value
Base Model	qwen2_vl_7b
Epochs	[3, 8, 20]
Batch Size	1
Learning Rate	1.0×10^{-4}

the risk of overfitting caused by excessive training and the problem of insufficient loss function convergence due to inadequate training rounds, so as to achieve a dynamic balance between model training efficiency and generalization ability. The experimental results for different numbers of training rounds are organized in Section 4.

2.3 Inference

2.3.1 Inference pipeline

**Figure 4** Inference pipeline

2.3.2 Post-processing

This study found two issues in the action sequences output by the fine-tuned model during experiments: Inconsistent action naming with the skill library: For example, the generated instruction "navigate to the black table" deviates from the skill library entry "navigate to the table 1". Mismatched action naming and sequence numbers: For instance, the skill "navigate to the table 1" corresponds to sequence number 10 in the skill library, but the generated instruction maps to number 12. In practical applications, although the logical structure of VLM-output instructions is correct, semantic mapping deviations from downstream code cause system parsing failures, increasing task failure rates. Existing evaluation systems misjudge task success rates by ignoring such execution barriers, affecting model performance assessment and optimization. To address this, we introduce a post-processing mechanism for model-output instructions: by constructing a large language model task judge, we perform semantic parsing and correction based on available skills in the library, converting flawed instructions into standardized formats recognizable by downstream programs. This eliminates semantic mismatches between instructions and execution, reducing task failure risks and enhancing the accuracy of success rate evaluation and model application efficiency.

3 Experiments

3.1 Pre-training Baseline Model Performance

Table 4 Pre-training Baseline Model Performance

Task Difficulty	Zero-shot	Few-shot
Base	4%	14%
Common Sense	0%	8%
Complex Instruction	2%	16%

The table 4 shows the task performance of the base qwen2_vl_7b model before training.

3.2 Post-training Model Performance and Experimental Conclusions

3.2.1 The performance of the fine-tuned qwen2_vl_7b model can approach that of the GPT-4o model

When the number of training epochs was 20, and the input only contained the user instruction and observation image, and the output only contained the action sequence, the experimental results of the fine-tuned qwen2_vl_7b and GPT-4o are compared in the table 5.

Table 5 Performance Comparison of Fine-tuned qwen2_vl_7b and GPT-4o

Task Difficulty	Fine-tuned qwen2_vl_7b	GPT-4o
Base	84%	86%
Common Sense	36%	38%
Complex Instruction	40%	42%

From the table 5, it can be concluded that with appropriate training parameters, the performance of the fine-tuned 7b model can approach that of the GPT-4o model.

3.2.2 The more concise the output of the fine-tuned model, the higher the success rate in complex tasks

When the number of training epochs was 20, and the input data included system-level prompts and examples, the experimental results after training with the v0 and v1 datasets respectively were as shown in table 6.

Table 6 Performance Comparison with and without Reasoning in Model Output

Task Difficulty	v0 (Output includes reasoning)	v1 (Output excludes reasoning)
Base	75%	74%
Common Sense	20%	30%
Complex Instruction	12%	16%

As can be seen from the table 6, the performance of models trained on v0 and v1 datasets is nearly identical on the base task set. However, on the common sense and complex instruction task sets, the model trained on the v1 dataset performs better. This suggests that when the output does not contain text such as visual reasoning and only includes the core action sequence to be executed, the training outcome is better than that of a dataset that includes reasoning content.

3.2.3 When the number of training epochs is high, the more concise the input to the fine-tuned model, the higher the task success rate

When the number of training epochs was 20 and the output only contained the action sequence, the experimental results were as shown in Table 7.

Table 7 Performance Comparison of Zero-shot (v1) vs. Few-shot (v2) Inputs

Task Difficulty	v1 (Zero-shot)	v2 (Few-shot)
Base	84%	74%
Common Sense	36%	30%
Complex Instruction	40%	16%

When the number of training epochs was 8 and the output only contained the action sequence, the experimental results were as shown in Table 8.

Table 8 Performance Comparison of Zero-shot (v1) vs. Few-shot (v2) Inputs (8 Epochs)

Task Difficulty	v1 (Zero-shot)	v2 (Few-shot)
Base	77%	62%
Common Sense	12%	16%
Complex Instruction	28%	18%

In the tables, "zero-shot" refers to the results from training with the v1 dataset, while "few-shot" refers to the results from training with the v2 dataset. The difference between them is whether system-level prompts and examples were added to the training data input. From the two tables above, it can be seen that adding examples slightly decreased the success rate across all difficulty levels. This phenomenon indicates that when the number of training epochs is large, the more concise the input to the model, the higher the task success rate.

3.2.4 When the number of training epochs is low, including example prompts in the input improves task success rate

When the number of training epochs is small, providing more content in the model's training input tends to yield better results. Table 9 are the experimental results for when the number of training epochs was 3 and the output only contained the action sequence.

Table 9 Performance Comparison of Zero-shot (v1) vs. Few-shot (v2) Inputs (3 Epochs)

Task Difficulty	v1 (Zero-shot)	v2 (Few-shot)
Base	46%	54%

3.2.5 The more training epochs for the fine-tuned model, the higher the task success rate

When the model's training input did not include example prompts and the output only contained the action sequence, the experimental results for different numbers of training epochs are as shown in Table 10:

From the table, it is clear that as the number of training epochs increases, the success rate shows a significant improvement.

4 Future work

- Train VLMs using online reinforcement learning algorithms

Table 10 Model Performance Across Different Training Epochs

Task Difficulty	Epochs=20	Epochs=8	Epochs=3
Base	84%	77%	46%
Common Sense	36%	12%	–
Complex Instruction	40%	28%	–

In the existing work rl4vlm [2], the authors embedded the simulation environment into a game scenario, and the performance of a trained qwen 7b model surpassed that of GPT-4o zero-shot.

- Develop offline reinforcement learning algorithms for training VLMs

Compared to online reinforcement learning, this can reduce engineering development effort. By referencing the preference datasets of DPO and utilizing the task progress and sub-goal reward information from each step in the existing dataset, combined with some offline reinforcement learning methods, data can be used more efficiently.

5 Conclusion

Building upon the work of EmbodiedBench, this research used CoT technology to optimize prompts and collected a small amount of high-quality data to fine-tune the qwen2_vl_7b model. Ultimately, it achieved a success rate comparable to the commercial model GPT-4o across various task difficulty scenarios. This study also set up multiple control experiments during the fine-tuning process to investigate the effects of the number of training epochs and the complexity of the input and output content in the training dataset on the final results.

References

- [1] Rui Yang, Hanyang Chen, Junyu Zhang, Mark Zhao, Cheng Qian, Kangrui Wang, Qineng Wang, Teja Venkat Koripella, Marziyeh Movahedi, Manling Li, Heng Ji, Huan Zhang, and Tong Zhang. Embodiedbench: Comprehensive benchmarking multi-modal large language models for vision-driven embodied agents, 2025. URL <https://arxiv.org/abs/2502.09560>.
- [2] Simon Zhai, Hao Bai, Zipeng Lin, Jiayi Pan, Peter Tong, Yifei Zhou, Alane Suhr, Saining Xie, Yann LeCun, Yi Ma, et al. Fine-tuning large vision-language models as decision-making agents via reinforcement learning. *Advances in neural information processing systems*, 37:110935–110971, 2024.
- [3] Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyang Luo, Zhangchi Feng, and Yongqiang Ma. Llamafactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, Bangkok, Thailand, 2024. Association for Computational Linguistics. URL <http://arxiv.org/abs/2403.13372>.